

Multi-Layer LLM Router

Cascade Intelligence Architecture
Generated May 06, 2026

This document defines the architecture for a cost-efficient, multi-layer LLM routing system. Prompts are routed through progressively more capable (and expensive) models. Each layer's output is judged by the next layer up — not by a fixed external judge. If all models fail, the **user becomes the final judge**, providing feedback that is fed back into the pipeline.

1. System Overview

The system has three LLM layers and one human layer. Each layer executes the prompt using its assigned model. The **next layer up acts as judge** — it evaluates whether the response fully answered the user's intent. Escalation only happens when the judge deems the response insufficient.

Layer	Role	Model (default)	Judge	Cost
1 — Local	First attempt	Ollama / qwen2.5:7b	Layer 2 (Haiku)	Free
2 — Fast	Fallback	Claude Haiku 4.5	Layer 3 (Sonnet)	Low
3 — Smart	Complex tasks	Claude Sonnet 4.6	User (human)	Medium
4 — Human	Ultimate fallback	You	—	Free

■ All models and judge assignments are configurable in router.yaml — swap any layer without touching code.

2. Cascade Judge Design (Key Innovation)

The judge is **not fixed**. Each layer is evaluated by the model one step above it. This means a stronger model always evaluates a weaker model's work — which is both cheaper and more reliable than using a fixed external judge for everything.

Layer executes	Judge	Judge prompt (simplified)
Layer 1 → response	Layer 2 (Haiku)	"Did this fully answer the question? YES / NO"
Layer 2 → response	Layer 3 (Sonnet)	"Did this fully answer the question? YES / NO"
Layer 3 → response	User (human)	Dashboard shows response + asks: Satisfied? (Yes/No)
User gives feedback	— (loop closes)	Feedback injected into Layer 3 as additional context

Why this works: Haiku is smart enough to know when a 7B local model missed the point. Sonnet is smart enough to know when Haiku was too shallow. The user is the only one who truly knows if Sonnet's answer solved their real problem. Each judge has exactly the right level of intelligence for what it is evaluating.

3. Full Decision Flow

Before calling the judge (which costs tokens), the signal detector scans the raw response text for failure markers. This costs nothing and catches obvious failures instantly.

■ *Signal detector runs first — judge only called if signals are found. Most good responses never reach the judge.*

Before any model is called, the pre-classifier estimates prompt complexity from keywords. This determines the starting layer — avoiding wasted calls to Layer 1 for tasks that are obviously complex.

■ Keywords are fully configurable in router.yaml. Add your own domain-specific trigger words.

When Layer 3 (Sonnet) fails the signal check, the response is shown to the user with two actions in the dashboard. The user's feedback is never discarded — it is injected as additional context for the retry, making the next attempt smarter.

Multi-Layer LLM Router — Architecture Document

3. Reject	User clicks <code>X Not helpful</code> → dashboard shows text field: 'I
4. Feedback	User types feedback → appended to original prompt as addit
5. Retry	Layer 3 (Sonnet) re-runs with enriched prompt → response s
6. Loop	Steps 2-5 repeat until user accepts or abandons the prompt

■ *User feedback is also saved to the agent's memory — improving future pre-classifier routing for similar prompts.*

7. Configuration (router.yaml)

Everything is controlled from a single YAML file. No code changes needed to swap models, disable layers, or change thresholds.

```
layers: - name: local type: ollama model: qwen2.5:7b enabled: true max_tokens: 1024 judge: fast
# judged by the 'fast' layer - name: fast type: anthropic model: claude-haiku-4-5-20251001
enabled: true max_tokens: 2048 judge: smart # judged by the 'smart' layer - name: smart type:
anthropic model: claude-sonnet-4-6 enabled: true max_tokens: 4096 judge: user # judged by the
human routing: signal_threshold: 1 # bad signals before calling judge judge_max_tokens: 50 #
judge only outputs YES/NO complexity: layer1_keywords: [list, show, status, check, what,
describe] layer2_keywords: [write, fix, create, explain, compare, find] layer3_keywords:
[architect, debug, refactor, design, review, why]
```

8. File Structure

```
agents/ ■■■ llm-router/ ← shared module (all agents import this) ■ ■■■ router.py ←
LLMRouter – main entry point ■ ■■■ classifier.py ← PreClassifier – keyword routing ■ ■■■
evaluator.py ← SignalDetector + MiniJudge ■ ■■■ router.yaml ← all config lives here ■ ■■■
layers/ ■ ■■■ base.py ← abstract BaseLayer ■ ■■■ ollama.py ← Ollama HTTP client ■ ■■■
anthropic.py ← Anthropic SDK + prompt caching ■ ■■■ workspace-agent/ ← imports LLMRouter
■■■ docker-manager-agent/ ← imports LLMRouter ■■■ agent-orchestrator/ ← imports LLMRouter
■■■ docs/ ■■■ llm_router_architecture.pdf ← this document
```

9. Cost Comparison

Assuming 100 user prompts per day across all agents:

Scenario	Distribution	Est. monthly cost
No routing (Sonnet all)	100% Sonnet	~\$45 / month
Simple routing	60% Haiku · 40% Sonnet	~\$18 / month
With Ollama layer	50% Local · 30% Haiku · 20% Sonnet	~\$7 / month
This architecture	40% Local · 35% Haiku · 25% Sonnet	~\$5 / month

■ *Estimates based on Anthropic pricing May 2026. Actual savings depend on prompt length and tool call frequency.*